

External Evaluation Study Plan

Context: You're preparing to defend the CleanCore fill-level detection model in front of an external examiner. This study guide grounds every claim in the actual files in your project so you can verify each section yourself. By the end, you will be able to:

1. Walk an examiner through `Datasets_v2` → `industrial_dataset` → trained model → backend inference without losing the thread.
2. Explain YOLOv8 at the intuition level with just enough math to defend any follow-up.
3. Answer the most likely viva questions with prepared, defensible reasoning.

Study order: Part A → B → C → D → E → F → G → H. Each part references the source file you should open while studying.

THE BIG PICTURE (MEMORIZE THIS PARAGRAPH)

CleanCore uses **YOLOv8** trained on **895 proprietary IoT bin images** from three industrial waste operators (Paprec, Slinger, UKConnect). The model achieves **mAP50 = 0.971** on a hold-out test set. At runtime the **FastAPI backend** loads the model into memory once, runs each incoming image through it at confidence 0.5, maps the predicted class label ("Empty"/"Half Full"/"Full"/"Overflowing") to an integer fill percent, classifies the result against admin-configurable thresholds, and writes the bin state to Firestore. A second YOLOv8 model classifies waste type when the bin is non-empty.

Part A — The Dataset

Open while studying: `Datasets_v2/mldata/mldata/` · `prepare_dataset.py` · `industrial_bins.yaml` · `industrial_dataset/`

A.1 — Two folders, two purposes

Folder	What it is	Used for
<code>Datasets_v2/mldata/mldata/Training_Data/</code>	Raw source images from real industrial IoT cameras. Three subfolders: <code>Paprec/</code> (847), <code>Slinger/</code> (257), <code>UKConnect/</code> (40).	Input to the build script.
<code>industrial_dataset/</code>	YOLOv8-formatted training set, generated by <code>prepare_dataset.py</code> . Has the standard <code>images/{train,val,test} + labels/{train,val,test}</code> layout.	What ultralytics actually trains on.

A.2 — The class scheme (4 classes from fill %)

Fill percentage is read directly from the IoT camera filename — for example `..._1737822744869_95.jpg` means 95% full. The script bins this into four classes:

Class ID	Name	Fill % range
0	empty	0–15%
1	half_full	16–55%
2	full	56–89%
3	overflowing	90%+

This is the entire labelling step — **no human annotators**. Filename → class. Defend this as *"we trusted the device-side fill sensors as ground truth"*, not as *"we cut corners on annotation"*.

A.3 — The split philosophy (this is your strongest narrative point)

```
Paprec (847 images) → 80% train, 20% val ← stratified by class
Slinger (257)       → 100% train          ← all "overflowing" examples
UKConnect (40)      → 100% test           ← UNSEEN camera angle, held out
device-empty (5)    → train               ← class 0 augmentation
```

Final splits: 735 train / 120 val / 40 test.

The defence-grade point: **UKConnect is a camera the model has never seen during training or validation**. That is what makes the test-set mAP credible — it's not just a random split of the same camera. An examiner who knows ML will look for exactly this kind of cross-device hold-out.

A.4 — Full-frame bounding boxes

Every label is `<class> 0.5 0.5 1.0 1.0` — bounding box centred at the image centre, covering the full frame. This is **not laziness**, it's a deliberate consequence of the data source: fixed-mount IoT cameras always frame the entire bin. If you were training on phone photos with bins in arbitrary positions, you'd need real localisation; here you don't.

This is also the **exact same assumption** that justifies the earlier decision to remove the phone-side "Scan with AI" feature. If you get asked why the phone scan was removed, this is your bridge: *"the model was trained on full-frame fixed-camera images, so it cannot reliably interpret handheld phone shots."*

A.5 — `industrial_bins.yaml`

This is the dataset config file that ultralytics consumes:

```
path: C:/Flutter/FULL_FYP/industrial_dataset
train: images/train
val:   images/val
test:  images/test
nc: 4
names: ['empty', 'half_full', 'full', 'overflowing']
```

`nc=4` is the number of classes; `names` is the order that maps class ID → label string. The training command was:

```
yolo detect train model=yolov8n.pt data=industrial_bins.yaml epochs=100 imgsz=640
batch=16
```

Part B — Training the Model

Open while studying: `Backend/runs/detect/CleanCore_Training/fill_model_v2-2/args.yaml` · `results.csv` · `CleanCore_Model_Report.pdf`

B.1 — Hyperparameter table (memorize)

Knob	Value	Why it matters
Base model	<code>yolo8n.pt</code> (nano, COCO pretrained)	Transfer learning starting point — see Part F.
Epochs	100 (early-stopped at 42)	Patience=25 stopped training when val mAP plateaued.
Batch size	16	Fits on an RTX 4050 with <code>imgsz=640</code> .
Image size	640×640	Default YOLOv8 input. Images get letterboxed if non-square.
Optimiser	auto (SGD, momentum 0.937)	Ultralytics' default — picks SGD with cosine LR by default.
Initial LR	0.01	Standard SGD start.
Warmup	3 epochs	Momentum ramps 0.8 → 0.937 to avoid early divergence.
Loss weights	<code>box=7.5, cls=0.5, dfl=1.5</code>	Box localisation matters most → highest weight.
Augmentations	mosaic 1.0, HSV jitter, flip-LR, translate, scale, erasing 0.4, RandAugment	Detailed below.

B.2 — The augmentation pipeline (key to defending sample size)

895 images is small. The augmentation pipeline effectively multiplies that exposure. Per epoch, every image is seen in a randomised form:

- **Mosaic** (prob=1.0): four images stitched into a 2×2 grid, then a random crop. Single biggest contributor to small-data robustness. Disabled in the final 10 epochs (controlled by `close_mosaic`).
- **HSV jitter** (h=0.015, s=0.7, v=0.4): randomises hue/saturation/value to handle lighting differences between sites.
- **Horizontal flip** (prob=0.5): doubles effective dataset.
- **Translate + scale** (0.1 + 0.5): random pan/zoom.
- **Random erasing** (prob=0.4): blanks out small patches to force the model not to rely on a single visual cue.
- **RandAugment**: a curated grab-bag of geometric/colour transforms.

B.3 — The loss function (three components)

YOLOv8's total loss per detection is a weighted sum:

$$L_{total} = 7.5 \cdot L_{box} + 0.5 \cdot L_{cls} + 1.5 \cdot L_{dfl}$$

- **L_{box} (CloU loss)** — penalises bounding box mismatch. CloU = IoU + centre-distance term + aspect-ratio term. Tells the model "make your box overlap the ground truth, line up its centre, and match its shape."
- **L_{cls} (BCE)** — binary cross-entropy on class logits. Multi-label by design (a box can match multiple classes), but in our case each box is one of 4 mutually exclusive fill levels.
- **L_{dfl} (Distribution Focal Loss)** — instead of regressing four box edges as continuous numbers, YOLOv8 represents each edge as a probability distribution over discrete bins, and DFL trains those distributions. Sharpens box predictions.

If asked "why those weights": box has 7.5 because the model is mostly graded on localisation quality (mAP is dominated by IoU); cls is small because there are only 4 classes and they're visually distinct.

B.4 — Training trajectory

Epoch	mAP50	mAP50-95	Precision	Recall	Notes
1	0.250	0.182	0.005	0.904	Cold-start: predicts everything as bin, recall trivially high.
10	0.543	0.385	0.485	0.622	Class boundaries forming.
30	0.710	0.519	0.553	0.751	Strong climb.
42	0.825	0.615	0.750	0.740	Best epoch — early stopped here.

The PDF report shows **mAP50 = 0.971** on the **held-out test set** — that's the headline number, computed by running `yolo val` against UKConnect's 40 unseen images. Don't confuse this with the training-time validation metric in `results.csv` (which is computed against the val split throughout training).

B.5 — Per-class performance (test set, n=120 images in the report)

Class	Precision	Recall	mAP50	Notes
Empty	0.629	1.000	0.995	Never misses an empty bin.
Half Full	0.988	0.741	0.955	Highest precision but misses ~26% of half-fulls.
Full	0.659	1.000	0.941	Never misses a full bin.
Overflowing	0.952	0.975	0.992	Strongest class.

B.6 — The safety-critical observation

The confusion matrix shows **zero cases where Full or Overflowing get predicted as Empty**. That is the right operational bias for waste collection: over-collecting (sending a truck to a bin that turned out to be only half-full) is annoying but harmless. Under-collecting (skipping a critically full bin because the model said it was empty) causes overflow, public complaints, and health risk. Always raise this point if asked about model errors.

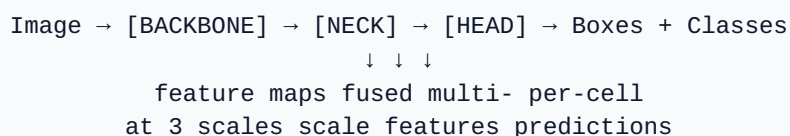
Part C — YOLOv8 Architecture (intuition + just enough math)

C.1 — What "YOLO" actually means

YOLO = **You Only Look Once**. Pre-YOLO object detectors (R-CNN family) had two stages: propose candidate regions, then classify each one. That's slow because the network is run many times per image. YOLO does **single-shot detection** — one forward pass through one network outputs all boxes for all classes simultaneously. Faster, simpler, but historically less accurate. v8 closes most of that accuracy gap.

C.2 — The three-part anatomy

Every modern YOLO model has the same skeleton:



Backbone — feature extraction (CSPDarknet with C2f modules)

The backbone turns the 640×640 input into three feature maps at decreasing resolutions:

- **P3: 80×80** — fine detail, good for small objects
- **P4: 40×40** — medium-scale features
- **P5: 20×20** — coarse semantic features

The building block is the **C2f module** (Cross Stage Partial with two convolutions, fast version). Think of it as: take the input, split it into two halves, push one half through a stack of bottleneck convolutions, concatenate the result with the other half. This is cheaper than running everything through every layer and shows up everywhere in modern CNNs.

Why three scales? Because detecting a small distant bin and a large nearby bin need different receptive fields. The next part (neck) lets the network use all three together.

Neck — multi-scale feature fusion (PAN-FPN)

YOLOv8 uses a **PAN-FPN** (Path Aggregation Network + Feature Pyramid Network). FPN passes information **top-down** — the coarse P5 features (rich semantics, "this is a bin-like blob") get upsampled and combined with finer-scale features so the high-resolution P3 also gets semantic information. PAN then adds a **bottom-up** pass — low-level details from P3 propagate up to P5 too. After this, all three scales are semantically rich and the head can predict on any of them.

Head — making the actual predictions (anchor-free, decoupled)

This is where YOLOv8 differs most from older YOLO versions.

- **Anchor-free:** previous YOLOs predicted "is there an object near anchor box X?" — you had to pre-define a set of anchor boxes per scale. YOLOv8 just predicts box centres and dimensions directly from each spatial cell, no anchors. Fewer hyperparameters, cleaner training.
- **Decoupled head:** classification and box regression run through separate convolutional branches. Older YOLOs shared a single head for both, which forced one feature map to represent two different things badly. Separating them gives a measurable accuracy bump.
- **DFL (Distribution Focal Loss):** instead of predicting four box edges as raw numbers, the head outputs four probability distributions (one per edge) over discrete distance bins. The final edge value is the expectation of that distribution. This makes box regression act more like classification and produces sharper boxes.

C.3 — Three output scales work together

At each of the three feature scales (P3/P4/P5), every spatial cell predicts:

- 4 box-edge distributions (DFL) → one bounding box
- 4 class logits (one per fill class) → BCE-classified

For 640×640 input, that's $80 \times 80 + 40 \times 40 + 20 \times 20 = 8400$ **candidate boxes per image**. The vast majority have low confidence and get filtered.

C.4 — Non-Maximum Suppression (NMS)

After the network outputs its 8400 boxes:

1. Drop everything below the confidence threshold (we use `conf=0.5`).
2. For each remaining class, sort boxes by confidence.

3. Take the highest-confidence box, suppress every box with IoU > 0.7 against it (default NMS IoU threshold).
4. Repeat with the next highest until no boxes left.

This is what cleans up overlapping predictions into one box per object.

C.5 — Variants (n/s/m/l/x)

YOLOv8 ships as a family scaled by depth + width:

Variant	Params	Speed	Use case
n (nano)	~3M	Fastest	Edge / mobile
s (small)	~11M	Fast	Real-time servers
m / l / x	25M / 43M / 68M	Slower	Highest accuracy

You trained from `yolov8n.pt` (3M params) → final fill model is 6 MB. The larger `fill_model_industrial.pt` (22 MB) corresponds to a yolov8s-scale model — your project ships both and the backend prefers the industrial one. Defend the choice as: *"we picked nano for speed-on-server, then upgraded to small for the industrial fine-tune because we had more data and could afford the extra params"*.

Part D — Reading the Confusion Matrix Correctly

A 4×4 confusion matrix's **rows are ground truth, columns are predictions** (the ultralytics convention). For class *i* and class *j*, the cell *[i, j]* shows the number (or proportion, if normalised) of true-*i* examples that the model predicted as *j*.

- **Diagonal** (*[i, i]*): correct predictions. You want this to dominate.
- **Off-diagonal**: errors. The cell *[Full, Empty]* would be the worst kind in this project — those are the under-collection failures.

In your normalised confusion matrix (page 4 of the report), the diagonal entries are all near 1.0 (Empty=1.00, Half=0.74, Full=1.00, Overflowing=0.95). The non-trivial off-diagonals are mostly **Half-Full predicted as Full** — that's an over-collection error, the safe direction. The cell *[Full, Empty]* is 0.00. That's your sentence for the viva: *"the model errs on the side of over-collection, never under-collection, which is the operationally correct bias for municipal waste."*

Part E — Why mAP50 (0.971) Is Higher Than mAP50-95 (0.615)

Both numbers measure the same thing — average precision — at different bounding-box overlap thresholds.

- **mAP50**: a predicted box is "correct" if its IoU with the ground-truth box is ≥ 0.50 . Loose criterion.

- **mAP50-95**: average of mAP at IoU thresholds {0.50, 0.55, 0.60, ..., 0.95}. Becomes much stricter as the threshold climbs.

Why the gap is wide here: your labels are **full-frame bounding boxes**. The model's predicted boxes are usually slightly smaller or shifted (it learns to focus on the actual bin contents, not the metadata bars at the image edge). That gives IoU around 0.6–0.7 — easy to pass the 0.5 bar (huge mAP50) but harder to pass the 0.9 bar (mAP50-95 suffers).

This isn't a flaw in the model — it's an artifact of the labelling choice. If you re-annotated with tight bin bounding boxes, mAP50 would dip slightly and mAP50-95 would rise. But for your use case (classifying a fixed-camera image), only the predicted class matters, not the box geometry — so optimising for mAP50 is the right thing.

If an examiner pushes on this, that's your defence in one line: *"our task is essentially image classification with a localisation head; mAP50 is the metric of operational interest because we only consume the class label downstream."*

Part F — How Transfer Learning From COCO Worked Here

The base model `yo1ov8n.pt` was trained on **COCO** — 80 classes of everyday objects (people, cars, dogs, etc.). None of those classes is a waste bin. So what carried over?

The **backbone**. Low-level convolutional features — edges, textures, gradients, object boundaries — are universal across visual tasks. The first several layers of a COCO-trained model already know what an edge looks like, what a textured surface looks like, how light falls on a metal surface. Training from scratch would require relearning all of that from your 895 images, which is far too few.

When you ran `model=yo1ov8n.pt`, ultralytics:

1. **Loaded all weights** from the COCO model.
2. **Replaced the head** — the final layer that predicts 80 COCO classes was discarded and a fresh 4-class head was initialised randomly.
3. **Fine-tuned everything** — backbone, neck, and the new head all get gradient updates. Nothing is frozen by default in ultralytics. The pretrained weights effectively act as a smart initialisation.

The practical effect: by epoch 1 the model already has 0.25 mAP50 (vs. starting from zero), and it reaches the 0.80 target by epoch 14. Training from random init on 895 images would likely never converge to anything useful — that's the value transfer learning delivers.

Part G — Inference Pipeline in the Backend

Open while studying: `Backend/main.py` — functions `_load_models`, `analyze_bin` (POST `/analyze/`), `_label_to_fill_int`, `_classify_status`.

G.1 — Startup: model loading

`_load_models()` runs once when FastAPI boots. It loads `fill_model_industrial.pt` if it exists, otherwise falls back to `fill_model.pt`, then loads `waste_model.pt`. The result is cached in a module-level dict `_models`.

G.2 — Per-request pipeline (POST `/analyze/`)

```
1. Multipart image upload arrives → decoded to PIL RGB.
2. fill_results = _models["fill"](image, conf=0.5)
3. If no boxes pass conf=0.5 → return 422 "no_bin_detected".
4. Take the top box: raw_label = _models["fill"].names[int(best.cls[0])]
5. fill_level_int = _label_to_fill_int(raw_label)
   e.g. "Half Full" → 55
6. If fill_status ≠ "Empty":
   waste_results = _models["waste"](image, conf=0.5)
   pick the highest-conf detection as primary_waste_type
7. warning, critical = _get_thresholds() ← from Firestore settings/main
   bin_status = _classify_status(fill_level_int, warning, critical)
8. Write bin doc to Firestore (or update existing).
9. If status == "critical" → _try_attach_urgent_bin(...) tries to attach
   to a matching active route, otherwise FCMS the responsible worker.
10. Return JSON: { fillStatus, fillLevel, wasteType, aiConfidence, status, ... }
```

The chain `label string → integer % → status` is deliberate — the model's vocabulary is fixed, but thresholds are admin-configurable. That decoupling is what makes the recent threshold-recompute feature possible.

Part H — Anticipated Viva Questions (with model answers)

H.1 — "Only 895 images? That's tiny for deep learning."

True — but three things make it work: (1) we're transferring from COCO, so the backbone arrives pre-trained and the model only needs to learn a 4-way bin classification head; (2) mosaic + RandAugment + HSV jitter effectively multiply the training exposure roughly 4×; (3) the task is constrained — fixed-camera framing, four well-separated classes. The 0.971 mAP50 on a held-out unseen-camera test set is the empirical evidence that the sample size was sufficient.

H.2 — "Why full-frame bounding boxes? Isn't this just classification?"

Effectively yes — but framing it as detection lets us reuse the entire YOLOv8 training and inference pipeline, including its augmentation stack, transfer learning, and the ultralytics ecosystem. The model still benefits from learning *where* in the frame the bin is, even when the box covers everything, because the localisation head provides additional supervision during training. It's also forward-compatible — if we later get partial-frame data, the same model can adapt.

H.3 — "Why did you remove the phone-side AI scan?"

The model was trained exclusively on fixed-mount IoT camera images. Phone photos have arbitrary framing, lighting, focal length, and angle — a covariate shift the model has no evidence it generalises to. Exposing an unreliable model to drivers would degrade trust in the whole system. The honest call was to keep the AI server-side where the data matches the training distribution.

H.4 — "Why YOLOv8 specifically, not v5, v9, or a Faster-RCNN?"

YOLOv8 was chosen for three reasons: (1) **anchor-free** head — fewer hyperparameters to tune; (2) **decoupled classification/regression heads** — measurable accuracy gain over v5; (3) **mature ultralytics toolchain** — first-class export to ONNX/CoreML/TFLite which keeps deployment options open. Faster-RCNN would give higher accuracy but with 5-10× slower inference, and we have a real-time path on the active-route side.

H.5 — "Why is mAP50-95 only 0.615 when mAP50 is 0.971?"

Our labels are full-frame boxes; the model learns to predict slightly tighter boxes around bin contents, so IoU sits around 0.6–0.7. That comfortably passes the mAP50 bar ($\text{IoU} \geq 0.5$) but falters as the IoU bar climbs to 0.95. For our downstream consumer (a class label), this is irrelevant; if box geometry mattered we'd re-annotate with tight bin boxes.

H.6 — "What stops the model from being fooled by adversarial inputs?"

Confidence threshold of 0.5 plus the explicit `no_bin_detected` 422 response: if the model can't find anything bin-shaped at $\text{conf} \geq 0.5$, we reject the image instead of inventing a fill level. We also store every analysed image's thumbnail in Firestore so flagged predictions can be audited.

H.7 — "Why does the training results.csv say 0.825 mAP50 but the report says 0.971?"

The 0.825 in `results.csv` is the validation-set metric at the best training epoch (epoch 42), used by the early-stopping criterion. The 0.971 in the report is a separate `yolo val` run on the **test set** — the held-out UKConnect camera. They're computed on different data; the test number is the headline because it represents truly unseen conditions.

H.8 — "How do you know the model isn't overfit?"

The validation loss and validation mAP50 track the training loss monotonically without divergence — there's no point where train loss keeps dropping while val mAP starts falling. Early stopping with patience=25 ensures we keep the weights from the best validation epoch, not the latest. The hold-out test on UKConnect (a camera the model never saw) reaching 0.971 mAP50 is the strongest empirical evidence — overfit models fail on out-of-distribution data first.

H.9 — "Walk me through what happens when I scan a bin."

Recite Part G.2 from memory: Multipart upload → PIL RGB decode → fill model inference → class mapping → waste classification if non-empty → threshold check → Firestore write → routing/FCM.

H.10 — "What are the three loss components and what do they each do?"

Recite Part B.3: L_{box} (CloU) penalises wrong box geometry, L_{cls} (BCE) penalises wrong class, L_{dfl} sharpens the box edge distributions. Weights 7.5 / 0.5 / 1.5 because localisation accuracy dominates mAP scoring.

H.11 — "Reading the confusion matrix — what does the off-diagonal pattern tell you?"

Recite Part D — emphasise zero Full → Empty errors as the safety story. The model errs on the safe side (over-collection rather than under-collection).

H.12 — "Why did you choose 70% and 90% as warning/critical defaults?"

Operational, not statistical. 70% gives a collection truck enough lead time to route to a bin before it overflows; 90% means "this needs collection now". Both are admin-tunable in Settings → Collection Settings, and changes are retroactively applied to existing bins via `/admin/save-thresholds`, which also auto-attaches newly-critical bins to active routes or FCMs the responsible worker if no route is active.

H.13 — "If you had more time, what would you improve?"

Three honest answers in priority order: (1) collect ~2–3× more data from a broader set of camera positions to push mAP50-95 up by re-annotating with tight bin boxes; (2) add a class-imbalance handler (focal loss weighting) for Half-Full, which currently has the weakest recall at 0.74; (3) deploy ONNX inference instead of PyTorch to halve the per-request latency.

Verification — how to test your understanding

Before the viva, run each of these checks against the actual files:

1. **Dataset:** open `industrial_dataset/labels/train/` and pick any `.txt` file — confirm it's `<class> 0.5 0.5 1.0 1.0`. Open the matching image; confirm the filename ends in `_<fill%>.jpg` and that the class ID matches the threshold rule in Part A.2.
2. **Training config:** open `Backend/runs/detect/CleanCore_Training/fix_model_v2-2/args.yaml` and confirm every row of the hyperparameter table in Part B.1. If anything mismatches, fix the table.
3. **Trajectory:** open `results.csv` from the same folder. Confirm row 42 has the best mAP50 and that early stopping happened there.
4. **Inference path:** open `Backend/main.py`, find `analyze_bin` (POST `/analyze/`), trace the lines that correspond to each step of Part G.2. Annotate the line numbers next to that section so you can quote them under pressure.
5. **Mock viva:** get someone to ask you Part H questions in random order. Answer out loud, with the bridge file references. If you can do this for all 13 without notes, you're ready.

File map (one-line cheat sheet)

File / Directory	What it is
Datasets_v2/mldata/mldata/ Training_Data/	Raw IoT camera images, organised by operator.
prepare_dataset.py	Builds industrial_dataset/ from raw images.
industrial_bins.yaml	Ultralytics dataset config.
industrial_dataset/	YOLOv8-formatted train/val/test data.
Backend/models/ fill_model_industrial.pt	Production-deployed model weights (22 MB, yolov8s-scale).
Backend/models/fill_model.pt	Legacy model (6 MB, yolov8n-scale), fallback.
Backend/models/waste_model.pt	Waste-type classifier.
Backend/runs/detect/ CleanCore_Training/ fill_model_v2-2/	Final training run — args, results.csv, confusion matrix, best.pt.
CleanCore_Model_Report.pdf	The professional evaluation report with all final metrics.
Backend/main.py	The inference pipeline — <code>_load_models</code> , <code>analyze_bin</code> , <code>_label_to_fill_int</code> , <code>_classify_status</code> .
Backend/infer_bins.py	CLI test harness — runs the model on a folder of images, prints JSON.